

Prompting for Existing Codebases

For developers extending, maintaining, and modernising legacy systems

Participants in the service industry are rarely building from scratch. More often the job is: understand what exists, find what's missing, plan what changes, and make the change safely. Every pattern here applies to an existing codebase.

1. Onboarding to an Unfamiliar Codebase

The situation: You've been handed a module or service you've never seen before.

Understand the entry point

```
@workspace Explain the entry point of this application.  
What is the request lifecycle – from the point the HTTP request arrives  
to the point a response is returned? Walk through the layers.
```

Map the domain

```
@workspace List all the domain entities this codebase models.  
For each, describe:  
- What it represents in business terms  
- Where it lives in the code  
- What other entities it relates to
```

Identify the patterns

```
@workspace What architectural patterns does this codebase follow?  
Give me concrete file paths as evidence for each pattern you identify.
```

Find the conventions

```
@workspace How does this codebase handle:  
1. Errors and exceptions  
2. Database access  
3. Logging  
4. Input validation  
5. Authentication and authorisation  
  
For each, give me an example file that demonstrates the pattern.  
I will follow these same conventions when I add new code.
```

2. Gap Analysis — Current State vs Desired State

The situation: You need to identify what's missing, inconsistent, or substandard.

Security gaps vs a standard

Acting as an application security engineer.

Review the following authentication implementation against OWASP Authentication Cheat Sheet 2024 and list every gap.

For each gap:

- Gap: what is missing or wrong
- Risk: what attack does this enable
- Priority: Critical / High / Medium / Low
- Fix: the specific change required

[paste the auth code or use @workspace for the whole module]

API contract gaps

The following is our OpenAPI specification for the /payments endpoint:

[paste spec]

Read the actual implementation in src/controllers/payment.controller.ts and list every discrepancy between the spec and the implementation:

- Endpoints in spec but not implemented
- Endpoints implemented but not in spec
- Response shapes that differ
- Validation rules that differ
- Error codes that differ

Compliance gap analysis (regulatory)

Acting as a PCI-DSS compliance engineer.

Review this payment processing flow and list every control that is required under PCI-DSS SAQ D but is either missing, only partially implemented, or implemented incorrectly.

Format each finding as:

- PCI-DSS Requirement: [requirement number]
- Current state: what the code does today
- Required state: what it must do
- Effort estimate: hours to remediate

Feature parity gap

Our legacy system (Java Spring) has the following features for user profile management:

[paste list of features or legacy code]

Our new system has this implementation:

[paste new code or describe the new system]

List every feature from the legacy system that is not yet present or not fully equivalent in the new system.

Categorise each as: missing entirely / partially implemented / behaviour differs.

Test coverage gap analysis

Acting as a QA engineer.

Review the following service class and identify every code path that has no corresponding test in the test file.

Service: [paste service]

Existing tests: [paste test file]

List uncovered paths in order of business risk, highest first.

For each, write the test case header (describe/it or @Test/displayName)

I should add, without implementing the body yet.

3. Impact Analysis — Before Making a Change

The situation: You need to change something. You need to know what breaks.

Blast radius of a change

@workspace I need to change the signature of the `calculateTax` method in TaxService from:

```
calculateTax(amount: number, countryCode: string): number
```

to:

```
calculateTax(amount: Money, jurisdiction: Jurisdiction): TaxResult
```

List every file in the codebase that calls this method.

For each, describe what the migration from the old signature to the new one requires.

Database schema change impact

I am adding a NOT NULL column `currency\_code VARCHAR(3)` to the `orders` table.

Analyse this codebase and identify:

1. Every query that INSERTs into the orders table – these will break
2. Every query that SELECTs from orders – these may need to return the new column
3. Every DTO or model class that maps the orders table – these need the new field
4. Any migration scripts that would conflict with this change

Dependency change impact

We are upgrading Spring Boot from 2.7 to 3.2.

Given this codebase, identify every breaking change we will encounter:

- `javax.*` imports that must become `jakarta.*`
- Deprecated APIs we are using that were removed in 3.x
- Configuration properties that changed
- Auto-configuration classes that changed behaviour

List them in the order I should fix them to get the application compiling.

Downstream service impact

@workspace Our OrderService publishes to the ``order.events.v1`` Kafka topic. I am changing the event schema to add a ``currency`` field and rename ``amount`` to ``amountCents``.

Which other services or consumers subscribe to this topic?

What changes do they need to make?

What is the safest migration sequence (dual-write, versioned topic, etc.)?

4. Feature Planning on an Existing Codebase

The situation: You have a feature request. You need to plan the implementation in a way that fits the existing architecture.

Generate an implementation plan that fits the codebase

@workspace I need to add rate limiting to all public API endpoints.

Before writing any code:

1. Describe how the existing middleware chain works in this codebase.
2. Where is the correct place to add rate limiting so it is consistent with how other cross-cutting concerns are handled here?
3. Write a step-by-step implementation plan that follows the existing conventions.
4. List every file I will need to create or modify.

Do not write any implementation code yet.

Technical design for a new feature

I need to add soft-delete to the User entity in this codebase. The codebase uses Spring Data JPA.

Design the technical approach covering:

1. Schema change required
2. How to filter soft-deleted records globally (so callers don't have to remember)
3. How existing queries will be affected
4. How to expose a restore capability
5. How audit logging should record deletions
6. What tests are needed

Base your design on the patterns already in this codebase – look at how other entities are structured and follow the same conventions.

Estimate complexity of a feature

Acting as a senior engineer who has read this codebase.

Estimate the effort to add multi-currency support to the checkout flow.

Structure your estimate as:

- What already exists that helps (reuse opportunities)
- What is missing and must be built
- What existing code must be modified
- Key risks and unknowns
- T-shirt size: S / M / L / XL
- Confidence: High / Medium / Low and why

Identify the right extension point

@workspace I need to send a notification after every successful payment.

Looking at how this codebase handles other post-payment side effects (e.g., order confirmation, inventory reservation), what is the correct extension point for adding this notification?

Should I:

- a) Add it directly in the payment service
- b) Use a domain event published by the payment service
- c) Use a database trigger or outbox pattern
- d) Something else that fits this codebase better

Justify your recommendation based on what is already in the code.

5. Legacy Code Modernisation

The situation: The code works but is outdated. You need to modernise safely.

Identify modernisation opportunities

Acting as a senior Java engineer specialising in modernisation.

Review this class and list every pattern that is outdated by modern Java 21 + Spring Boot 3 standards.

For each:

- What the current pattern is
- What the modern equivalent is
- Risk of the change: Low / Medium / High
- Whether behaviour will change or is purely stylistic

Order by: highest impact, lowest risk first.

Modernise incrementally (safe changes first)

I want to modernise this legacy service but cannot risk breaking it.

Categorise every change you would make into:

SAFE – purely stylistic, zero behaviour change (rename, extract variable, etc.)

LOW RISK – structural refactor with equivalent behaviour (extract method, introduce interface)

HIGH RISK – changes that touch business logic or external contracts

Give me only the SAFE changes first, as a list of specific edits I can make and commit independently with no fear of regression.

Replace an anti-pattern across the codebase

@workspace This codebase uses raw string concatenation to build SQL queries in multiple places. Find every occurrence.

For each occurrence:

- File and line number
- The current unsafe query
- The parameterised replacement using Spring's JdbcTemplate or JPA

Order by risk (those accepting user input first).

6. Technical Debt Mapping

The situation: You need to communicate the state of the codebase to your manager or client.

Generate a technical debt inventory

@workspace Acting as a senior engineer preparing a technical debt report for a client presentation.

Review this codebase and produce a technical debt inventory:

Area	Debt Description	Business Risk	Effort to Fix	Priority
-----	-----	-----	-----	-----

Categories to check:

- Security vulnerabilities
- Outdated dependencies with known CVEs
- Missing test coverage on critical paths
- Inconsistent error handling
- Undocumented public APIs
- Duplicated business logic
- Performance anti-patterns (N+1, missing indexes)
- Hardcoded configuration

Limit to the top 10 most impactful items.

Prioritise the debt

Here is our technical debt list:

[paste list]

Acting as a senior engineer, prioritise this list using the RICE framework (Reach, Impact, Confidence, Effort) from the perspective of a financial services company where reliability and compliance are paramount.

Give each item a RICE score and a recommended quarter to address it.

7. Generating Documentation for Undocumented Code

The situation: You inherited code with no documentation. The original author is gone.

Reverse engineer the API contract

Read this service class and generate an OpenAPI 3.1 specification for all its public endpoints.

Infer the request/response shapes from the code itself.

Where the behaviour is ambiguous, add a comment in the spec flagging it for human review.

Generate a module README

Read the following module and write a README.md that covers:

1. What this module does (one paragraph, plain English, no jargon)
2. How it fits into the overall system
3. Key classes and their responsibilities
4. How to add a new [entity type] following the existing pattern
5. Common pitfalls – things the code makes easy to do wrong
6. Dependencies – external services this module talks to

Write for an engineer joining the team with 2 years of Java experience but no knowledge of this codebase.

Explain a complex algorithm

Acting as a senior engineer mentoring a junior.

Explain what this algorithm does in plain English, step by step. Then explain WHY it was written this way – what problem does this specific approach solve that a simpler approach would not?

Then list any edge cases that are silently handled and would not be obvious from the code alone.

[paste algorithm]

8. Incident Investigation Prompts

The situation: Something broke in production. You need answers fast.

Root cause from stack trace

We have a production incident. Here is the stack trace:

[paste stack trace]

And here is the relevant code:

[paste code]

Work backwards from the exception to identify:

1. The immediate cause
2. The root cause
3. Whether there is a data or state condition that triggered this
4. A fix that addresses the root cause (not just catches the exception)
5. What we should add to prevent silent failure next time

Find all similar vulnerabilities

We just discovered that our UserService had an N+1 query problem in the `getUsersWithOrders` method – it was calling the database once per user in a loop.

@workspace Find every other method in this codebase that follows the same pattern: calling a repository or database method inside a loop over a collection.

List each occurrence with file, method name, and a suggested fix.

Summary: Prompt Patterns by Workflow

Workflow	Key prompt trigger
Understanding code	Explain the... Walk through the layers... What patterns does this follow?
Gap analysis	List every gap between X and Y... What is missing?... What does not conform to?
Impact analysis	What breaks if I change... Which files call... What downstream effects?
Feature planning	Where is the right place to add... Design the approach... What files will I touch?

Modernisation	What is outdated... Categorise changes by risk... Replace this pattern across...
Debt mapping	Produce a debt inventory... Prioritise using RICE... What are the top 10?
Documentation	Generate an OpenAPI spec... Write a README... Explain this algorithm...
Incident investigation	Work backwards from... Find all similar... What is the root cause?

The Three Questions to Ask Before Every Change

Before writing any code in an existing codebase, prompt with these three:

1. "How does this codebase currently handle [the concern I'm adding]?"
→ Find the pattern to follow.
2. "What will break if I change [what I'm about to change]?"
→ Understand the blast radius.
3. "What is the right place in this architecture to put this?"
→ Avoid creating new inconsistencies.

These three questions, answered by Copilot reading your actual codebase, replace the 30-minute "let me look around" exploration every developer does when touching unfamiliar code.